

AVL Tree

15 June 2026 19:06

What is an AVL Tree?

An AVL Tree is a self-balancing Binary Search Tree(BST) invented by *Georgy Maximovich Adelson-Velsky* and *Evgenii Landis*

AVL stands for: Adelson-Velsky and Landis

Like a BST:

- Left subtree contains smaller values.
- Right subtree contains larger values.

Additionally, AVL Tree maintains a balance condition:

For every node, the height difference between the left and right subtree must be at most 1.

This height difference is called the **balance factor**.

Balance Factor

Balance Factor(BF) of a node:

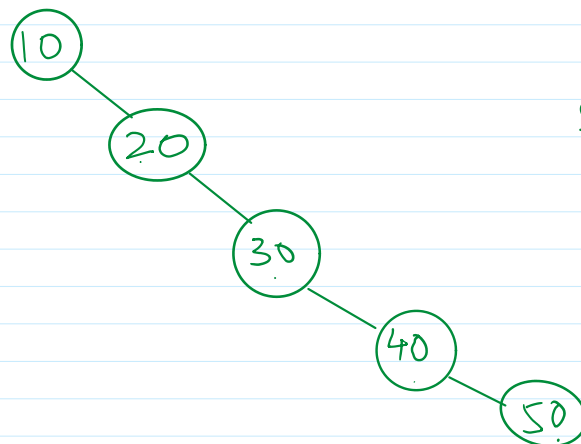
$$BF = \text{Height}(\text{Left subtree}) - \text{Height}(\text{right subtree})$$

Possible value:

Balance Factor	Status
-1	Balanced
0	Balanced
1	Balanced
< -1	Unbalanced
> +1	Unbalanced

Why AVL Tree?

10, 20, 30, 40, 50



Searching:

Best case = $O(\log n)$

worst case = $O(n)$

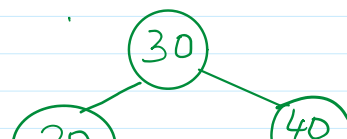
Average case = $O(n)$.



Performance is bad.

This becomes a skewed tree.

AVL Tree automatically

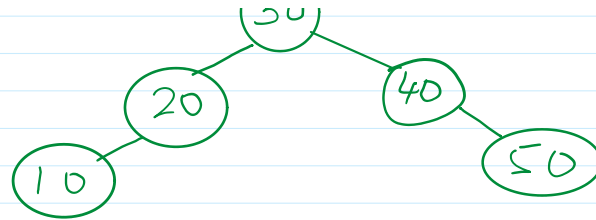


AVL Tree automatically balances itself:

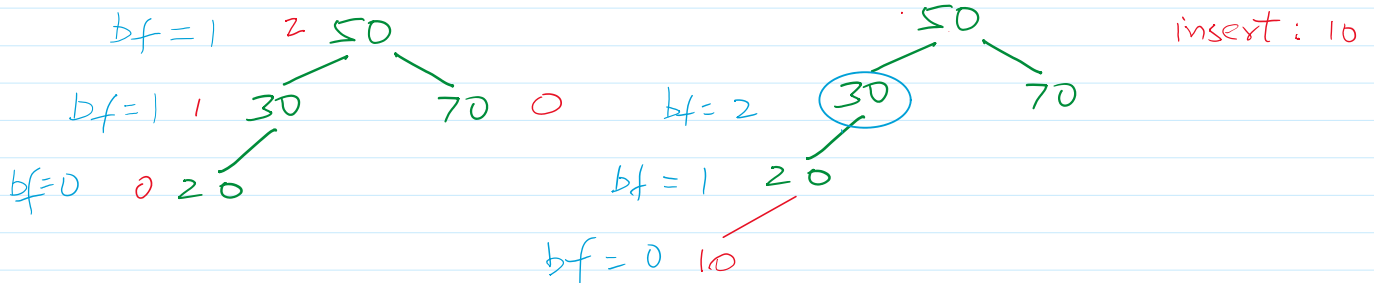
Search: $O(\log n)$

Insert: $O(\log n)$

Delete: $O(\log n)$



Height of a Node: Number of edges on the longest path from root node to leaf.



AVL Rotations

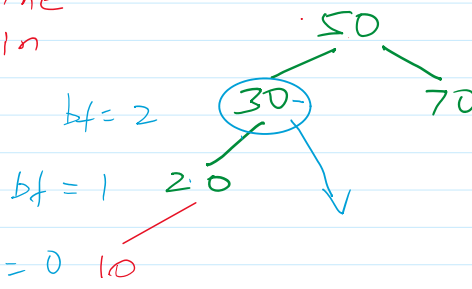
Whenever a node or subtree becomes unbalanced, AVL performs rotations:

There are four cases:

1. LL Rotation.
2. RR Rotation.
3. LR Rotation
4. RL Rotation

1. LL Rotation (Left-Left case): Occurs when insertion is made in the left subtree of the left child.

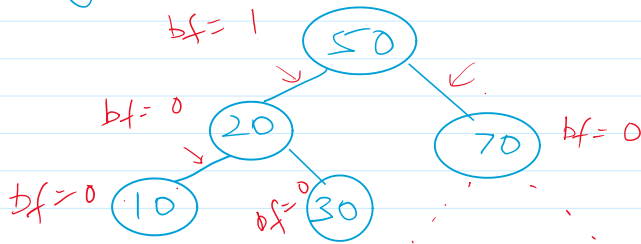
↓
we rotate the unbalanced part in right



Balance factor of 30 is +2. [Unbalanced].

Solution: Right rotation.

Solution: Right rotation.

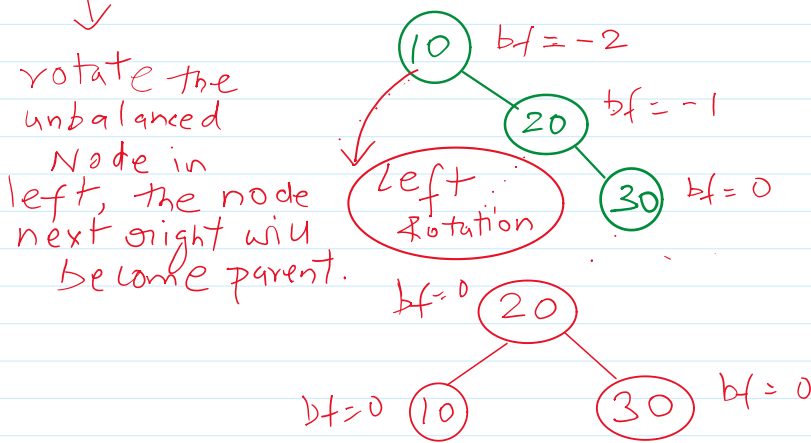


Now the tree is balanced.

2. RR rotation (Right Right Case):

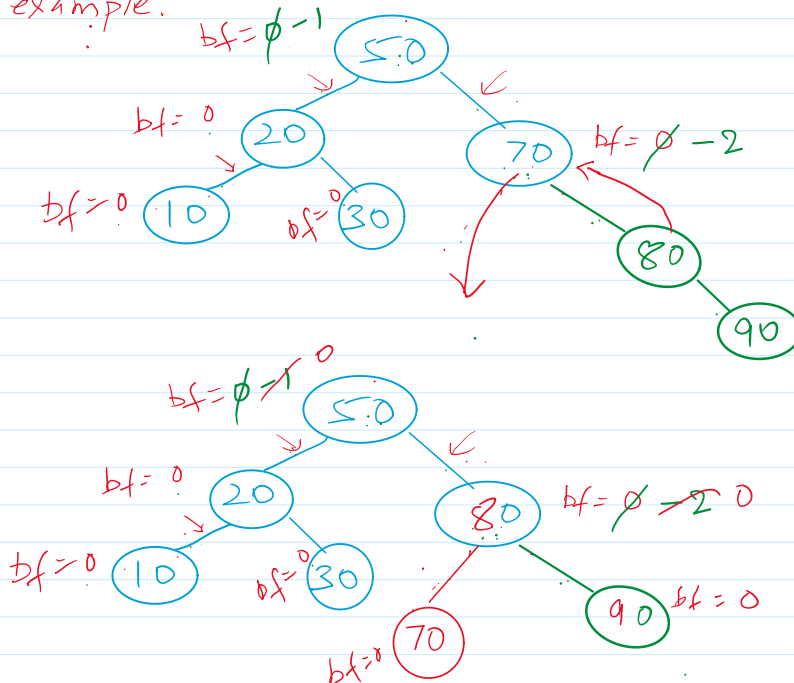
Occurs when insertion is made in right subtree of right child.

Insert 10, 20, 30

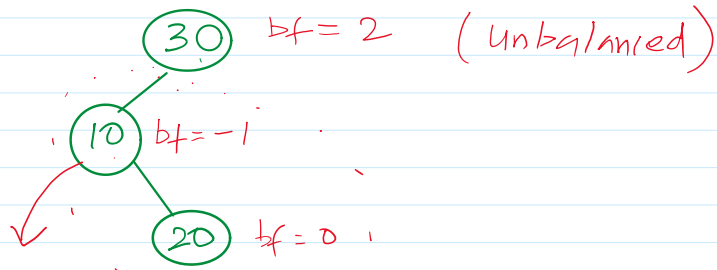


Another example:

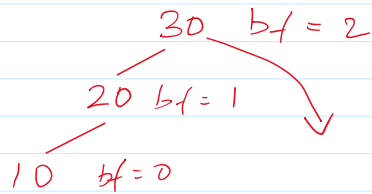
Insert 80 & 90



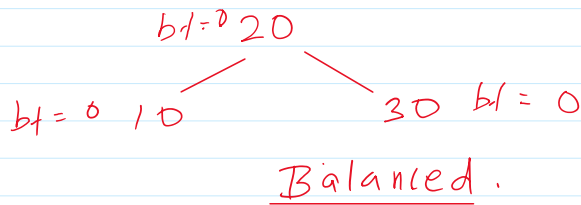
LR Rotation (Left-Right case) : When the last insertion is done on right, and BST is unbalanced.
 Insert: 30, 10, 20.



Step 1: Left rotation on 10.

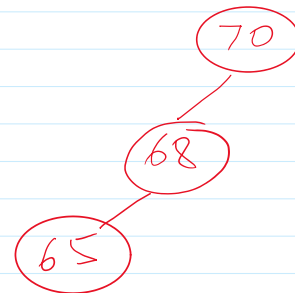
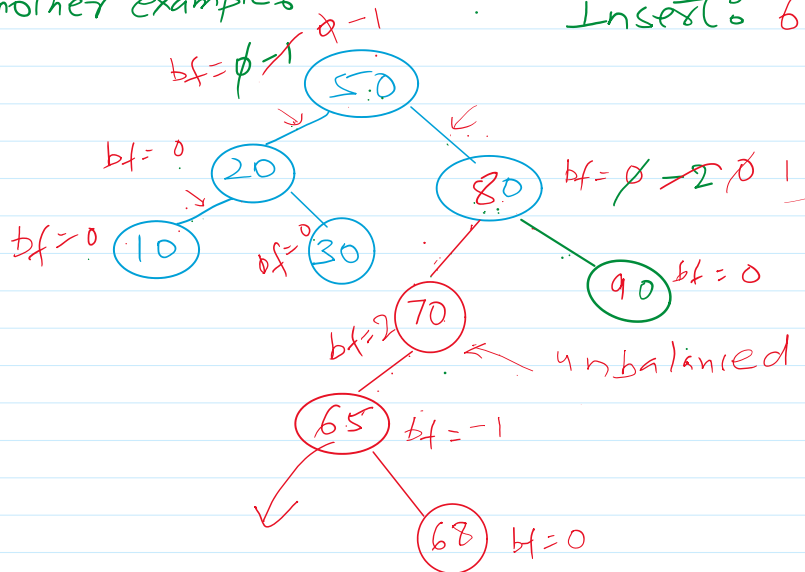


Step 2: Apply Right Rotation on 30.

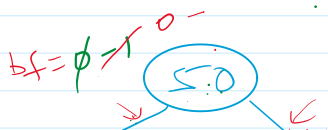


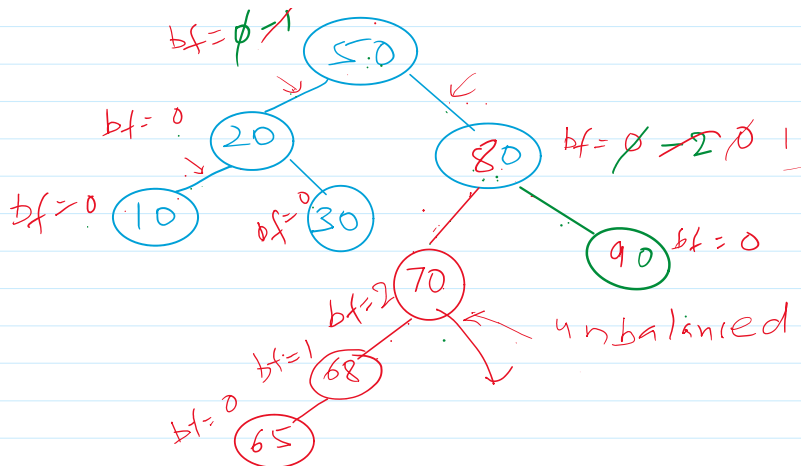
Another example:-

Insert: 65, 68

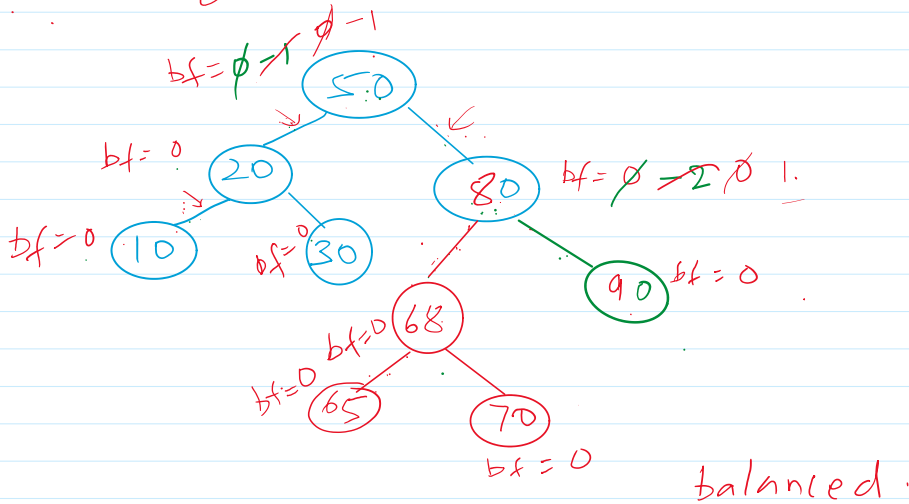


Step-1. left rotation.



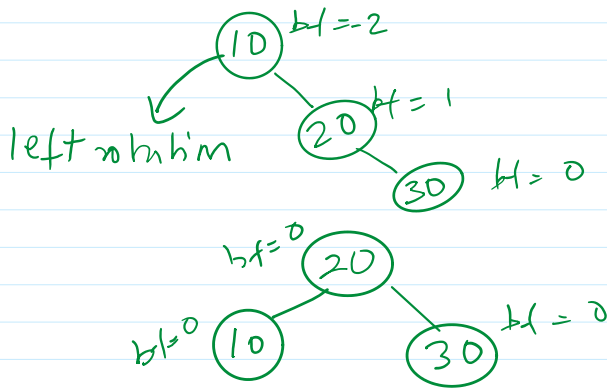
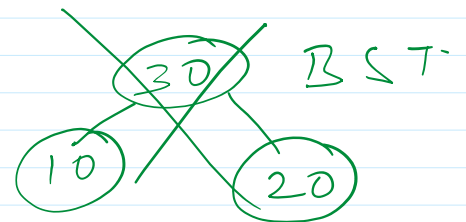
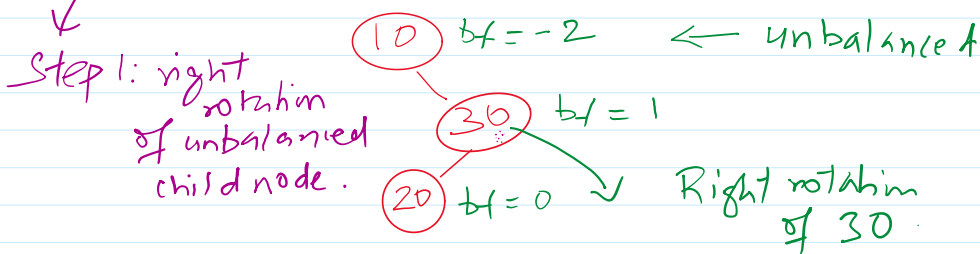


Step-2 Right Rotation:



RL Rotation: (Right-left Case) ← insertions.

Insert 10, 30, 20



Step-2, left rotation of unbalanced node.

balanced.

Another example:

(Right - left)
insert, 40, 35

